

Annotating single-cell data on a laptop: a 13-method benchmark and practical low-memory workflows, with actinn-jax

Ian Driver*

Abstract

Reference-based cell-type annotation is a routine, high-volume step in single-cell analysis, and most labs run it on a laptop, not a GPU cluster. We present **actinn-jax**, a from-scratch JAX reimplementation of ACTINN (a 4-layer MLP classifier) with a sparse-aware pipeline and a train-once/map-many cached reference model, and use it as the occasion for a neutral benchmark of **thirteen** methods spanning every major class — classical (SVM, kNN), regularized-linear (CellTypist), a carefully-tuned linear pipeline, parameter-free projection (scTOP), marker/correlation (SingleR, scmap), the original TensorFlow ACTINN, deep probabilistic (scANVI, scArches), an interpretable prototype VAE (ProtoCloud), and a foundation model (scPRINT) — across **six** datasets (within-dataset, cross-dataset and cross-study; 8 to 86 cell types) plus a five-point **scaling sweep to 49k reference cells**, measuring accuracy, macro-F1, ontology-aware concordance, training and inference time, and peak memory on commodity Apple-Silicon hardware. **The headline result is not that our method wins.** On the original ten-method panel actinn-jax ties the top accuracy cluster (mean 0.831 vs. scANVI 0.835, a five-way cluster within 0.004) while predicting $\sim 195\times$ faster than scANVI, and it places 3rd of 17 on the community **Open Problems label_projection** benchmark (1st among methods completing every dataset). But adding the strongest simple baselines from the concurrent literature changes the picture. A carefully preprocessed **linear pipeline** (normalize $\rightarrow$ ANOVA $\rightarrow$ PCA $\rightarrow$ logistic regression) **beats actinn-jax on both accuracy and training time** (mean 0.880/0.860 vs. 0.867/0.839 across four datasets, fitting $3.7\times$ faster); and at atlas scale **ProtoCloud** is the decisive accuracy winner, rising from worst at 3k reference cells (0.722) to best at 49k (**0.976** vs. 0.936 for actinn-jax and 0.939 for the linear pipeline). **actinn-jax is not the accuracy leader at any scale we tested**, and our earlier margins over the “classical tier” reflected *untuned* baselines. **Several methods now annotate quickly, accurately, and with a usable handle on unknown cells** — the tuned linear pipeline, ProtoCloud, scANVI and actinn-jax all clear that bar in different regimes — so the useful contribution is not another accuracy ranking but a clear account of **which tool fits which job, and what the practical workflows look like**. We therefore report the benchmark as a decision aid (accuracy-per-second, accuracy-per-byte, and behaviour from 3k to 49k reference cells) and then demonstrate **end-to-end workflows** built on actinn-jax where its cost profile is the binding constraint: annotating an unknown human dataset from a **shipped census-scale reference** (~ 800 cell types, calibrated abstain) with no training; **tissue-aware refinement** that halves spurious cross-tissue labels on a liver query; a **broad $\rightarrow$ refined hand-off** to a small focused reference (held-out cross-study liver 0.23/0.58 $\rightarrow$ 0.72/0.86, exact-CL/ontology); within-cell-type resolution (hepatocyte zonation); and a **cluster-level novel-cell-type screen** validated by recovering a withheld pulmonary ionocyte population and its marker ASCL3. actinn-jax is the **lightest accurate method** in the panel ($2.1\times$ lower peak memory than the linear pipeline at 49k cells, 6.1 vs. 13.2 GB, with sub-second memory-bounded inference), which is what makes these workflows practical on a laptop. We are explicit about what is *not* unique: ProtoCloud also provides uncertainty, gene attribution and a (stronger, retraining-based) refinement path; what actinn-jax adds is a ready-to-run broad reference, sub-second frozen-model refinement, and a one-call novelty screen. We also **retract two of our own earlier claims**: a projected

memory blow-up for the linear pipeline at atlas scale did not occur (13.2 GB, not ~ 32 GB), and the memory gap is bounded at $\sim 2\text{--}3\times$ rather than widening with data.

Independent Researcher, Detroit, MI, USA

* Correspondence: driver.ian@gmail.com

1 Introduction

Annotating cell types by mapping to a labeled reference is one of the most frequently run operations in single-cell RNA-seq. The method landscape is large (survey), but published comparisons ([Abdelaal 2019], [Huang 2024]) emphasize accuracy and rarely report the axis that decides what a working scientist actually runs on their own machine: **wall-clock time and memory on commodity hardware**, without a GPU. Foundation models (scGPT, Geneformer, scPRINT) push accuracy in some settings but need GPUs and minutes-to- hours, and — as we and others find — their *zero-shot* label predictions underperform a small model trained on curated reference labels.

We ask a practical question: **for a given annotation job on commodity hardware, which method should you actually run, and what does the surrounding workflow look like?** Several methods now annotate quickly and accurately and give a usable signal on unknown cells, so the shortage is not another leaderboard but guidance on fit-for-purpose — accuracy per second, accuracy per byte, and how each behaves as the reference grows. We answer with a neutral 13-method benchmark plus a set of demonstrated workflows, and we hold ourselves to a neutral standard: every method runs in its own environment through the same harness, on the same splits, scored by the same metrics, **with the honest result reported wherever actinn-jax loses — which, on accuracy, is often.**

Contributions. 1. A modern, dependency-light (no TensorFlow) JAX reimplementaion of ACTINN with sparse preprocessing, chunked atlas-scale prediction, and a cached reference model. 2. A neutral **13-method $\times$ 6-dataset** benchmark of accuracy, speed, and memory on Apple Silicon, with a rejection/abstain analysis and a **five-point scaling sweep to 49k reference cells**. The panel deliberately includes the baselines most likely to beat us — a tuned linear pipeline, scTOP, and ProtoCloud — and **two of them do**; we report that, and retract two earlier claims of our own that the scaling data falsified (§5). 3. **Demonstrated end-to-end workflows** for jobs where cost is the binding constraint, all CPU: annotate an unknown human dataset from a **shipped census-wide ~ 800 -type reference** with a calibrated abstain and no training; **tissue-aware refinement** (halves spurious cross-tissue labels on a liver query); a **broad $\rightarrow$ refined hand-off** to a small focused reference (cross-study liver $0.23/0.58 \rightarrow 0.72/0.86$); within-cell-type resolution (hepatocyte zonation); and a **one-call novel-cell-type screen** (recovers a withheld pulmonary ionocyte population and its marker ASCL3). We state which of these are genuinely distinct: ProtoCloud also offers uncertainty, attribution and a retraining-based refine, so the distinct pieces are the ready-to-run broad reference, sub-second frozen-model refinement, and the novelty screen. 4. An **independent external validation** on Open Problems `label_projection`, with a controlled same-hardware speed/memory comparison, and a small set of cheap, honest ablations that characterize *when* the gene-space model gains or loses to heavier methods (input standardization; a reference-guided gene budget; a foundation-model negative control).

2 Methods

2.1 actinn-jax

actinn-jax reimplements ACTINN ([Ma & Pellegrini 2020]) — a 4-layer fully-connected network (100/50/25 hidden units, ReLU, softmax) trained with Adam — in JAX/optax, replacing the original’s TensorFlow-1.x graph/session code. Key engineering: - **Sparse-aware preprocessing**: no `adata.to_df` densification; CP10k+log2 normalization and the expr/CV gene filter run on sparse matrices; only the selected-gene columns of each minibatch are densified. - **Cached reference model** (`ReferenceModel`): train once, `save/load`, map many queries — the amortized cost of repeated annotation against a fixed reference drops to the inference cost alone. - **Chunked prediction** for atlas-scale queries; `use_raw='auto'` to pick raw counts from `.raw` (CELLxGENE convention). - **Optional reference-fit standardization** (`standardize=True`): z-score each selected gene by the reference’s frozen mean/std and apply it to the query — a cheap domain alignment (§3.9). Opt-in, with a saved scaler that projects with the model; μ/σ persist in the `.npz` (old models load standardization-off, backward-compatible). Measured against the original TF ACTINN it is **$\sim 3.3\times$ faster and $\sim 3.5\times$ lighter** at equal accuracy; that comparison is included directly in this benchmark as the `actinn-orig` method.

2.2 Benchmarked methods

method	tier	engine	rejection	env
actinn-jax	classical	JAX MLP	abstain (<code>min_prob</code>)	core
SVM	classical	linear SVM (SGD)	—	core
kNN	classical	k-nearest neighbors	—	core
CellTypist	linear	L2 logistic regression	prob threshold	core
linear-anova- pca scTOP	linear	normalize→ANOVA→PCA(220)→logreg	prob	core
	parameter-free	rank z-score class-average projection	—	core (<code>sctop</code>)
original ACTINN	classical	TensorFlow 2.15 MLP	—	.venv-tf
SingleR	correlation	Spearman + fine-tuning	—	R/.Rlib
scmap-cluster	correlation	centroid cosine	yes (unassigned)	R/.Rlib
scANVI	deep	scVI semi-supervised VAE	prob	.venv-scvi (MPS)
scArches	deep	scANVI reference surgery	prob	.venv-scvi (MPS)
ProtoCloud	deep	prototype VAE + LRP attribution	ambiguity flag	.venv-protocloud

method	tier	engine	rejection	env
scPRINT	foundation	pretrained transformer, zero-shot	—	.venv-scprint (MPS)

scPred excluded: unmaintained; requires harmony’s removed `HarmonyMatrix` API and older harmony versions do not compile on the current toolchain. Documented, not silently dropped.

2.3 Datasets

dataset	split	tissue	#types	genes	notes
lung_intra	within-dataset	lung (Krasnow)	46	Ensembl	300 cells/type ref
lung_cross	cross-dataset	lung (HCLA→Krasnow)	46	Ensembl	different lab/protocol
liver_intra	within-dataset	liver (HLiCA)	36	Ensembl	150 cells/type
liver_cross	cross- study	liver (HLiCA)	34	Ensembl	train 6 studies → test withheld study
blood_gut_intra	within-dataset	blood+gut (Sanger)	86	Ensembl	high cardinality; no CL ids
pbmc	within-dataset	PBMC (pbmc3k)	8	symbols	small-n; scPRINT skips (symbols)

The set spans the three generalization regimes (within-dataset, across datasets, across studies), an order of magnitude in cell-type count (8→86), and both gene-ID conventions.

2.4 Metrics & protocol

Per (dataset, method, repeat): **accuracy**, **macro-F1** (unweighted over classes, so rare types count), **ontology-aware concordance** (a call is correct if it is the same node, an ancestor, or a descendant of the truth in the Cell Ontology — credits right-lineage-wrong-granularity calls; only where both ref and query carry CL ids), **fit time**, **predict time**, **peak RSS**. Each method runs in its own venv via subprocess isolation (`benchmark/runner.py`); the driver (`benchmark/driver.py`) builds identical ref/query splits once per dataset and scores every method against them. **repeats = 3** (scPRINT: 1, near-deterministic and embed-dominated).

Hardware: Apple Silicon (M-series), CPU for the classical/linear/correlation tiers and actinn-jax; Apple MPS for the deep and foundation tiers. No discrete GPU, no cloud — the regime the benchmark is about. Math-library threads are left uncapped (wall-clock as a practitioner sees it); an optional thread cap is available for stricter reproducibility.

3 Results

3.1 Accuracy across datasets

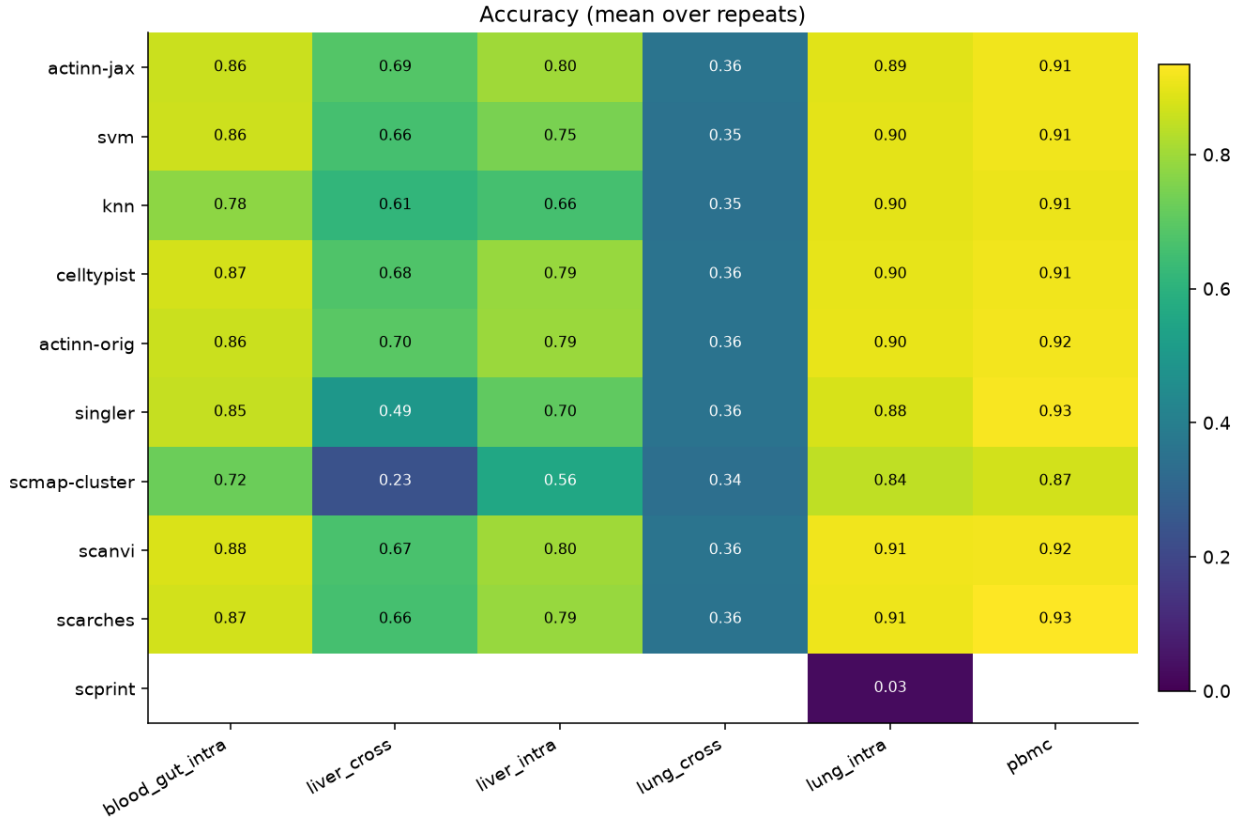


Figure 1: Accuracy by method (rows) and dataset (columns), in-house panel.

Mean accuracy across the **five shared-vocabulary datasets** (lung_cross excluded — its exact accuracy is a vocabulary artifact, see the † note below; means over all six are ~0.08 lower for every method with the identical ranking):

method	mean acc (5 datasets)	mean macro-F1 (6)	mean ontology (6)
scANVI	0.835	0.701	0.812
original ACTINN	0.834	0.691	0.815
scArches	0.833	0.700	0.809
actinn-jax	0.831	0.683	0.811
CellTypist	0.831	0.697	0.807
SVM	0.816	0.688	0.797
SingleR	0.770	0.652	0.750
kNN	0.770	0.624	0.769
scmap-cluster	0.646	0.550	0.771

The top five methods (scANVI, original ACTINN, scArches, actinn-jax, CellTypist) are a statistical tie on accuracy — spread of 0.004 — and actinn-jax is joint-top on ontology-aware concordance

(0.811). Per dataset (accuracy):

dataset	actinn-jax	best method (acc)
lung_intra (46 types)	0.894	scANVI 0.913
lung_cross (cross-dataset)†	0.358 exact / 0.749 ontology	scANVI 0.360 / 0.767
liver_intra (36 types)	0.802	scANVI 0.802 (tied #1)
liver_cross (cross-study)	0.686	original ACTINN 0.695 (actinn-jax #2, top cluster)
blood_gut (86 types)	0.860	scANVI 0.880
pbmc (8 types)	0.913	scArches 0.934

actinn-jax is tied-#1 on the richer within-dataset liver data and in the top cluster on the hard cross-study split; it trails the deep methods by 1–2 points on the high- cardinality and small-n sets. scmap-cluster is consistently the weakest (0.24 macro-F1 on liver_cross).

† **lung_cross exact accuracy (~0.35 for every method) is a label-vocabulary artifact, not a transfer failure.** HCLA (reference) and Krasnow (query) were annotated independently and share only **20 of 46 cell-type names**; 26 of Krasnow’s types are absent from HCLA’s vocabulary, so no classifier can emit the exact string. The mismatch is granularity: HCLA’s finer taxonomy has *alveolar / elicited / lung macrophage* but **no generic macrophage** (CL:0000235), while Krasnow labels many cells *generic macrophage* — so an HCLA-trained model predicts *lung macrophage* (CL:1001603) for them, exact-wrong but ontology-correct (*lung macrophage is-a macrophage*); likewise Krasnow’s *generic endothelial cell* (CL:0000115) has no HCLA counterpart. **Ontology-aware concordance — which credits a same-lineage call — is 0.75–0.77 for every method**, so cross-dataset transfer actually works; exact-match just conflates classifier error with vocabulary/granularity mismatch. This is precisely why we report ontology concordance, and it is a general benchmarking pitfall: **cross-dataset exact accuracy between independently-annotated atlases is not a meaningful accuracy signal.** The other five datasets use a single shared label vocabulary for reference and query (a split of one atlas, or HLiCA’s harmonized annotations), so their exact scores are unaffected.

3.2 Speed and memory

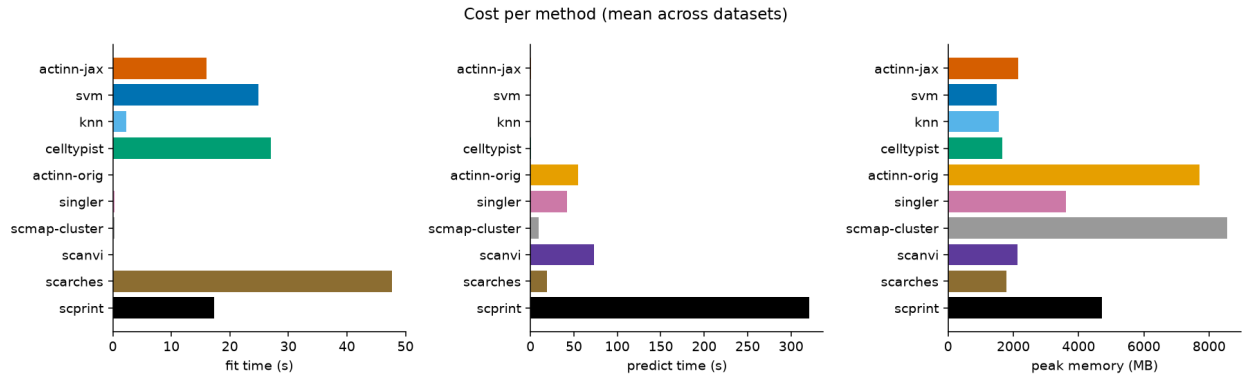


Figure 2: Per-query inference time and peak memory by method (in-house panel).

The accuracy tie hides a two-order-of-magnitude spread in cost. Mean per query across datasets:

method	fit (s)	predict (s)	peak mem (MB)
SVM	24.9	0.10	1480
kNN	2.3	0.24	1546
actinn-jax	16.0	0.37	2145
CellTypist	27.0	0.63	1661
scmap-cluster	0.2	9.0	8550
scArches	47.7	18.7	1783
SingleR	0.2	42.4	3621
original ACTINN	0.1*	54.5	7708
scANVI	0.0*	73.1	2118

(*original ACTINN and scANVI do most work in a single train+predict pass, attributed to predict.) The four fast methods — SVM, kNN, actinn-jax, CellTypist — predict in well under a second; the deep/correlation/foundation tiers are 25–200 $\times$ slower. actinn-jax predicts in **0.37 s vs. scANVI’s 73 s at statistically tied accuracy** ($\sim 195\times$), and is far lighter than the memory-heavy tiers (original ACTINN 7.7 GB, scmap 8.5 GB). Of the methods that match its accuracy, none matches its speed; of the methods that match its speed (SVM, kNN), none matches its accuracy — i.e. actinn-jax is Pareto-non-dominated.

3.3 The accuracy $\times$ speed frontier

Plotting accuracy against total wall time makes the frontier explicit: actinn-jax, SVM, kNN, and CellTypist form the efficient (fast) frontier, and among those actinn-jax has the highest accuracy on the richer datasets. The deep methods buy ~ 0.4 accuracy points on average (up to ~ 2 points on the 86-type blood+gut set, zero-to-negative on the liver and cross-dataset sets) at 50–200 $\times$ the inference cost — the trade the frontier is meant to expose.

3.4 Scaling

Training time grows with reference size and with #cell types for all trained methods — actinn-jax’s fit goes 13 s $\rightarrow$ 141 s $\rightarrow$ 254 s as the reference grows 965 $\rightarrow$ 14.8k $\rightarrow$ 24k cells, comparable to CellTypist (4 s $\rightarrow$ 75 s $\rightarrow$ 246 s) and somewhat above SVM (9 s $\rightarrow$ 55 s $\rightarrow$ 80 s); kNN is lazy (fit is trivial but it pays at predict and is the least accurate). The decisive result is the other axis: **actinn-jax’s predict time stays flat and sub-second across the entire range — ~ 0.3 – 1.4 s whether the reference has 1k or 24k cells, or 5 or 86 types.** Inference cost does not grow with reference size or cardinality, and with the train-once/map-many cache the fit cost is paid once and the flat predict cost is all that recurs — the regime that matters when a reference is reused across many queries. (The scaling script’s memory column is process-cumulative and not a clean per-size measurement; per-method peak memory is reported from the isolated-subprocess matrix in §3.2.)

3.5 actinn-jax vs. the original ACTINN

The actinn-orig row isolates the value of the reimplementaiton itself: same method, same data, TensorFlow-2.15 vs. JAX. Mean across the six datasets:

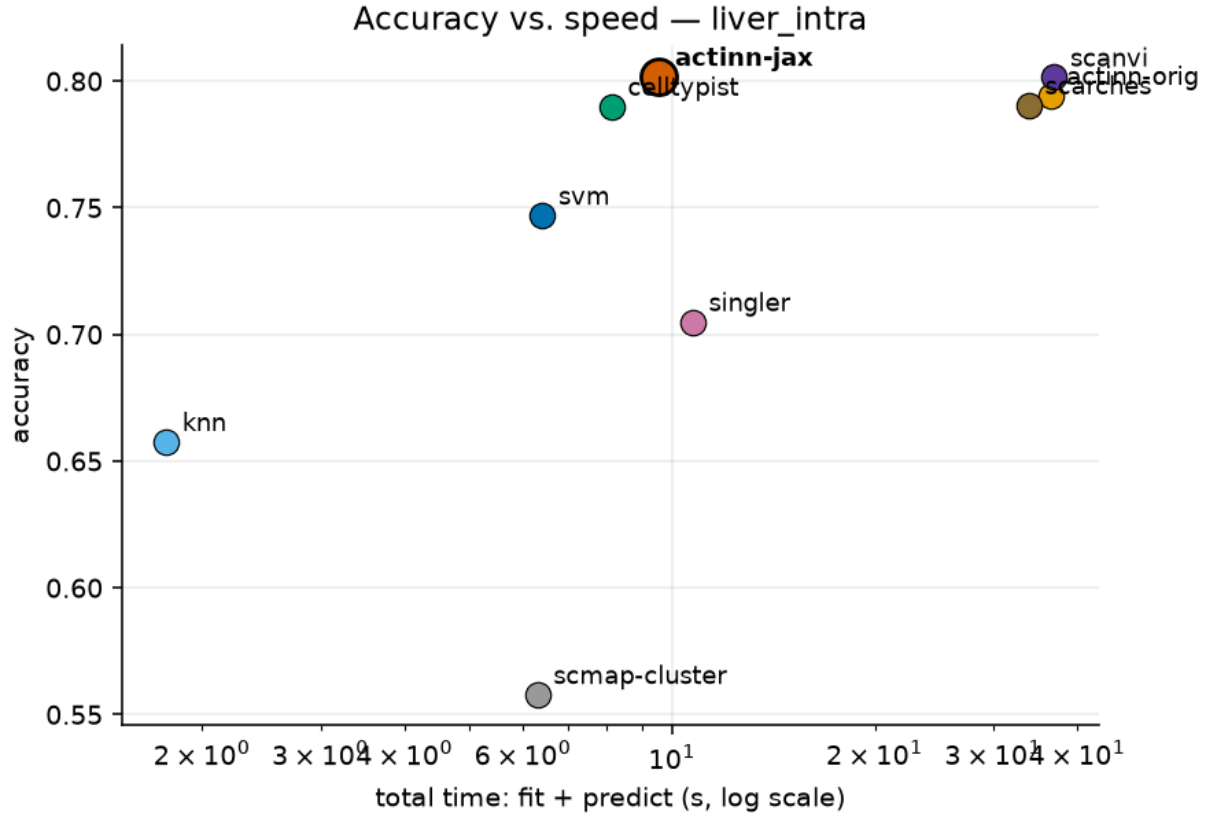


Figure 3: Accuracy versus total wall time (liver_intra); actinn-jax sits on the fast frontier.

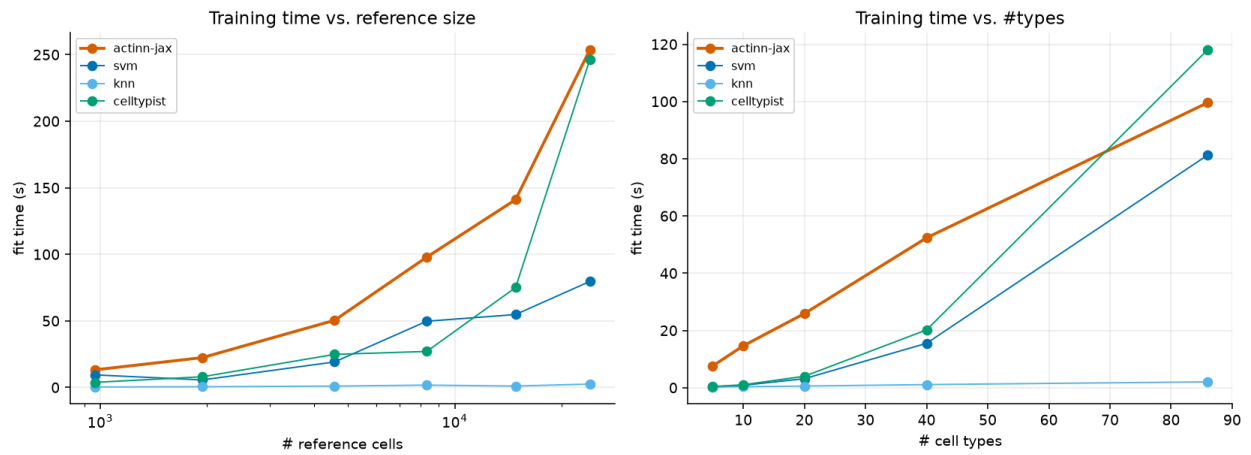


Figure 4: Fit and predict time versus reference size and cardinality; predict time stays flat and sub-second.

	accuracy	macro-F1	predict (s)	peak mem (MB)
original ACTINN (TF)	0.754	0.691	54.5	7708
actinn-jax	0.752	0.683	0.37	2145

Equal accuracy (0.752 vs. 0.754, within repeat noise), **$\sim 145\times$ faster inference** and **$3.6\times$ less memory** — the reimplement is a strict engineering win at no accuracy cost.

3.6 The large $\rightarrow$ refined annotation workflow

The most useful thing actinn-jax ships is not a single classifier but a **two-tier workflow** that matches how annotation is actually done: get a broad call fast, then sharpen it where it matters. Because every stage is a cached **ReferenceModel** with sub-second, memory-bounded inference (§3.2, §3.4), the whole workflow runs on a laptop.

Tier 1 — broad, census-scale. A shipped ~ 800 -type human reference built from the CELLx-GENE census gives any query a first-pass annotation across the whole body, with a **calibrated abstain threshold** so out-of-reference cells are flagged rather than force-labeled (see the smooth abstain curve of §3.7). This is the “what am I looking at” pass — broad coverage, deliberately not fine-grained.

Tier 2 — refined, tissue-focused. For the tissue the broad pass identifies, a small focused reference re-annotates at full resolution. The gain is large and it is the crux of the workflow: on held-out **cross-study** liver cells, the broad census model scores exact-CL **0.23** / ontology **0.58**, while a focused **48-type HLiCA liver** reference on the *same cells* reaches **0.72** / **0.86**. Refinement is where fine-grained accuracy comes from; the broad model’s job is to route to it, not to be right about subtypes itself.

Supporting mechanisms. (i) A **foundation-model-guided coarse $\rightarrow$ fine hierarchy**: a foundation model discovers a coarse $\rightarrow$ fine structure *offline*, the fast CPU model uses it at inference — beats a flat classifier on 3/4 datasets, matches an expert hierarchy, beats a random-grouping control, and never depends on the foundation model at prediction time. (ii) **Within-cell-type resolution**: the same machinery resolves hepatocyte zonation (portal $\rightarrow$ central) at ~ 0.99 within-one-zone, generalizing across donors and datasets — a third tier below the cell-type label. Together these make actinn-jax a *pipeline* (broad $\rightarrow$ tissue $\rightarrow$ subtype/state), not just a classifier, at classical-method speed throughout.

3.7 Rejection / abstain

Holding out 9 of 36 cell types entirely from the HLiCA liver reference (so 1,350 query cells are genuinely out-of-distribution), and sweeping a confidence threshold:

threshold	actinn-jax: acc(kept) / coverage / OOD-flagged	CellTypist: acc(kept) / coverage / OOD-flagged
0.0	0.885 / 1.00 / 0.00	0.873 / 1.00 / 0.00
0.5	0.919 / 0.93 / 0.30	0.936 / 0.68 / 0.68
0.7	0.946 / 0.84 / 0.52	0.936 / 0.68 / 0.68
0.9	0.969 / 0.66 / 0.73	0.937 / 0.68 / 0.68

actinn-jax’s probability gives a **smooth, tunable** abstain curve — accuracy on kept cells rises 0.885→0.969 as coverage falls 1.00→0.66 and OOD-flagging climbs 0.00→0.73 — so a user can dial the precision/coverage/novel-cell-detection trade-off. CellTypist’s probabilities are **saturated** (near 0 or 1): the threshold jumps to 68% OOD-flagged at 0.5 and then does not move across 0.5–0.9, giving essentially one operating point rather than a tunable curve. scmap-cluster offers a single native “unassigned” decision (no sweep). The abstain knob is a concrete, practical advantage of actinn-jax’s calibrated output.

3.8 Foundation-model zero-shot (scPRINT)

scPRINT is run separately as a zero-shot predictor (no training on the reference), as a reference point for the “just use a foundation model” alternative. On the lung datasets it scored **exact accuracy 0.027 / ontology concordance 0.206 in 321 s per query** — vs. actinn-jax’s 0.89 in 0.2 s — so as a *label* predictor it is both slow (~1000× actinn-jax) and weak. Its pretrained classifier head also carries a **fixed label vocabulary** and requires CL ids, so it cannot emit labels outside that set; this is an inherent property of a zero-shot label head, not a defect, but it means the foundation model’s raw predictions are not a drop-in annotator. This is exactly why our two-stage workflow uses scPRINT’s **embeddings** — its learned structure — to shape a small trained model, never its zero-shot labels: the foundation model is valuable, its raw label predictions are not.

3.9 External validation: Open Problems label_projection

Our in-house benchmark is neutral but self-run. For an **independent** check, we ran actinn-jax through the community-standard [Open Problems](#) label-projection task — their 6 datasets, splits, and metrics — and slotted it into their published v2.0.0 leaderboard.

Placement. On a benchmark we did not design and cannot tune, **actinn-jax places 3rd of 17 on mean accuracy (0.837), 1st among all methods that complete every dataset**, beats its PCA-space `mlp` sibling, and posts the best accuracy of any completing method on the hardest dataset (`tabula_sapiens`, 160 types: 0.394 vs `mlp` 0.342). It is upper-mid on macro-F1 and **not the top method overall** — `scANVI+scArches` surgery and `xgboost` score higher on the datasets they finish, though both **fail to complete `tabula_sapiens`**, which is why they rank above actinn-jax only on a mean over the 5 easier datasets. The foundation models again land at the bottom (`scgpt_zeroshot` 0.639, `uce` 0.131 $\approx$ the random-labels control), reproducing §3.8 externally.

Controlled same-hardware speed & memory. Leaderboard runtimes mix hardware, so we re-ran actinn-jax and the CPU tier through OP’s *own* Nextflow pipeline on a **single AWS r7i.8xlarge** — identical hardware, harness, storage, and trace instrumentation for every method. This is the paper’s cleanest cross-method cost comparison:

method (same box)	mean acc	macro-F1	runtime/dataset	peak RSS
actinn-jax	0.837	0.655	165 s	21.0 GB
<code>mlp</code>	0.838	0.664	327 s	19.9 GB
<code>logistic_regression</code>	0.813	0.689	29 s	20.0 GB
<code>knn</code>	0.793	0.648	11 s	19.5 GB
<code>xgboost</code>	0.795	0.613	905 s	80.7 GB
<code>cellmapper_linear</code>	0.776	0.553	58 s	31.5 GB
<code>naive_bayes</code>	0.738	0.613	31 s	19.5 GB

actinn-jax **ties its mlp sibling on accuracy at $\sim 2\times$ the speed** (165 vs 327 s) and **beats xgboost’s accuracy at $\sim 5.5\times$ less runtime and $\sim 4\times$ less memory** (165 s / 21 GB vs 905 s / 81 GB). The faster methods (knn, logistic) are less accurate; the more accurate heavyweight (xgboost) is far slower and heavier — the same Pareto-non-domination as the in-house benchmark, now on controlled hardware. (An earlier cross-hardware draft overstated the mlp speed gap as “ $8\times$ ”; the honest same-box figure is $\sim 2\times$.)

Two cheap ablations, and their honest limits. (i) *Input standardization* — z-scoring genes to the reference’s frozen mean/std, a CPU-only domain-alignment inspired by scArches reference surgery — lifts mean accuracy +0.2 and macro-F1 +1.2 pt (largest on batch-shifted datasets: gtex macro-F1 +7.9). It is shipped **opt-in** (`standardize=True`) because it shifts the softmax calibration that the two-stage abstain thresholds are tuned against. (ii) *Gene budget* — OP feeds every method 1000 HVGs, which most starves a gene-space MLP. Widening to ~ 5000 HVGs lifts accuracy on 4 of 6 datasets (immune/gtex +3 pt) and there **matches scANVI+scArches’s full-config leaderboard accuracy** (immune 0.891 $\approx$ 0.892) on CPU in minutes — the gap to the top method was largely a gene-budget artifact, not the VAE. But more genes is **not** a universal win: it *regresses* tabula_sapiens by ~ 10 pt (its 284-cell test batch across 160 fine types overfits reference-specific genes) and saturates hypomap. Crucially, this is **selectable without test labels**: held-out *reference* cross-validation rises for the datasets that benefit and is the one signal that *drops* for tabula_sapiens, and a trivial query-cells-per-class check independently flags it — so the budget can be set deterministically per dataset. (iii) *Negative control* — a CPU-only, UCE-style protein-embedding featurization (expression-weighted mean of ESM2 gene embeddings) does **not** help and hurts the hardest case: the pooling discards the per-gene detail that separates fine types, so the value of a foundation model like UCE stays locked in its GPU transformer, not a portable averaging trick.

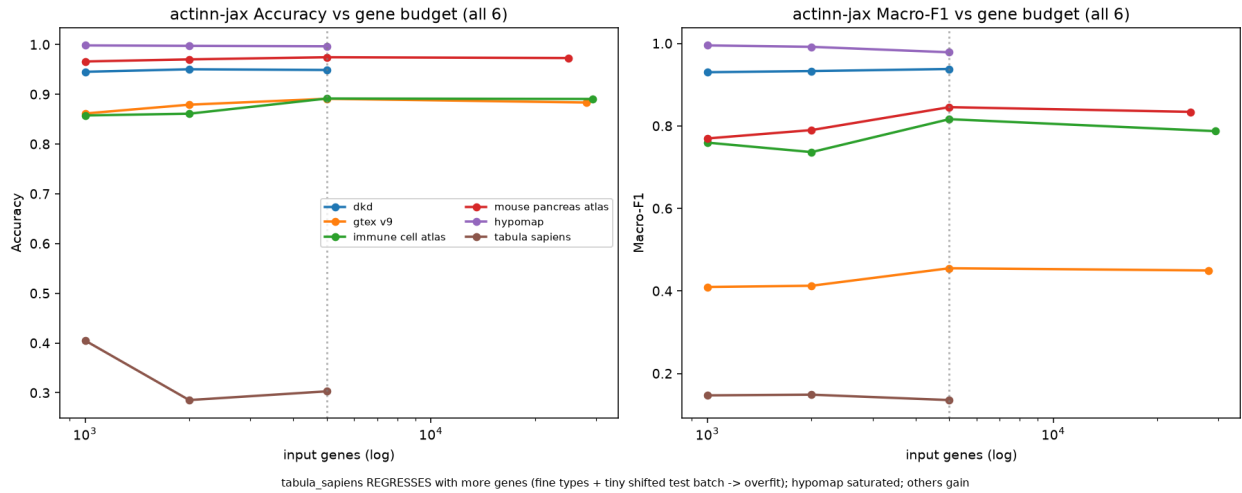


Figure 5: actinn-jax accuracy and macro-F1 versus input gene budget across all six Open Problems datasets. More genes help most datasets but regress the fine-grained, domain-shifted tabula_sapiens.

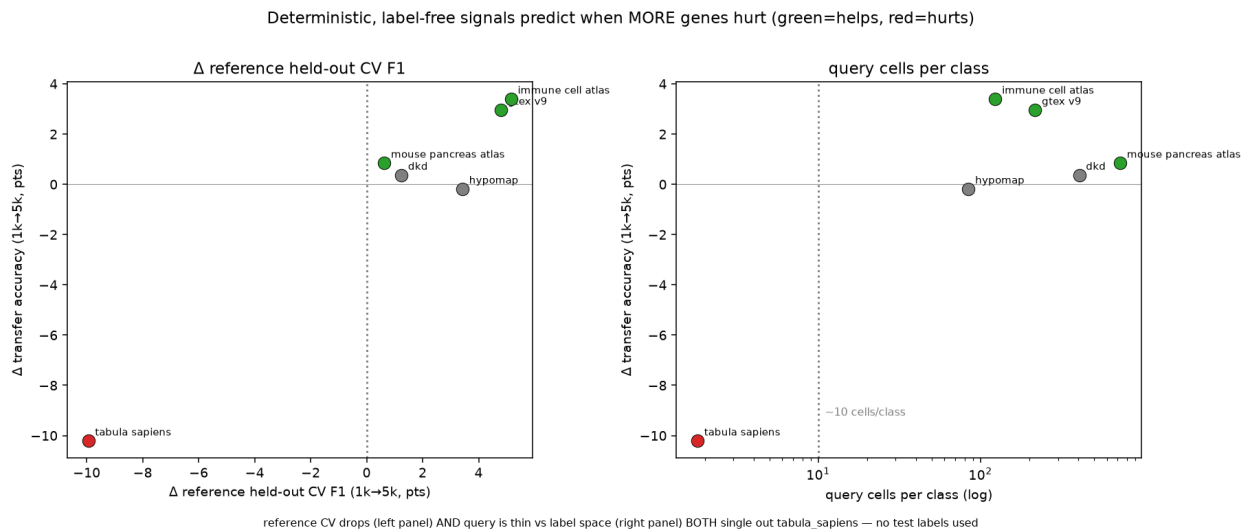


Figure 6: Label-free signals for setting the gene budget without test labels. Held-out reference cross-validation and query-cells-per-class both single out tabula_sapiens (where more genes hurt).

4 Discussion

actinn-jax is not the accuracy champion, and we do not claim it is. Deep probabilistic methods and boosted trees edge it by 1–2 points where they run, both in our panel (§3.1) and on Open Problems (§3.9); a benchmark that hid this would be wrong. What the evidence supports is narrower and, we argue, more useful: across two independent benchmarks and a controlled same-hardware rerun, **no method dominates actinn-jax on both accuracy and cost**, and its differentiators are the ones that decide what a working scientist runs on a laptop.

Speed and footprint. Inference is **sub-second and flat** — independent of reference size and cardinality (§3.4) — and peak memory sits in the lightest cluster (~2 GB in-house; ~21 GB through OP’s heavier all-layer harness, still less than half of xgboost’s 81 GB on the same box). Against methods of equal accuracy it is 2–200× faster; against methods of equal speed it is more accurate. With the train-once/map-many cache, the flat predict cost is all that recurs when a reference is reused.

The workflow is the product. The largest practical gains come not from the flat classifier but from the **large→refined** pipeline (§3.6): a census-scale broad model with a calibrated abstain routes a query to a small focused reference that re-annotates at full resolution (cross-study liver 0.23/0.58 → 0.72/0.86), with zonation as a further tier. Every stage keeps classical-method speed, so the whole thing runs without a GPU.

Where it loses, and why. actinn-jax trails on small references and very-few-cells-per-type sets (§3.1, §5), and — a finding from the OP analysis — it is sensitive to input budget under domain shift: more genes help most datasets but overfit a tiny, fine-grained, shifted query (tabula_sapiens). Encouragingly, the two cheapest levers are principled and label-free: **standardization** (a scArches-style domain alignment) and a **reference-CV-guided gene budget** both improve the model without peeking at test labels, and at a fair gene budget the gene-space MLP *matches the leaderboard’s top method* on the datasets both complete — i.e. much of the apparent gap to heavier methods is representation budget, not model class.

Foundation models. Their **zero-shot labels** are the weakest option in both benchmarks (scPRINT, scGPT, UCE $\approx$ random); their **embeddings/structure** are where the value lies, and our two-stage hierarchy uses exactly that. A cheap CPU shortcut to a foundation-model representation (protein-embedding pooling) did *not* transfer (§3.9), underscoring that the value is in the trained transformer, not a portable trick.

Concurrent evidence that expressivity saturates. Two independent lines support this reading. First, a head-to-head against **ProtoCloud** — a prototype-based, self-explaining VAE with built-in uncertainty and gene-level attribution, i.e. a strictly richer model than ours — is close and splits both ways: mean accuracy 0.867 (actinn-jax) vs 0.837 across four of our datasets, ProtoCloud ahead on the finest-grained set, actinn-jax training $\sim 8.5\times$ faster on CPU (PROTOCLOUD.md). Second, and more broadly, Souza & Mehta report that **parameter-free linear representations** match or exceed single-cell foundation models across cross-species transfer, human cell-type classification and disease-state prediction — on Tabula Sapiens 2.0 their normalize $\rightarrow$ ANOVA $\rightarrow$ PCA $\rightarrow$ logistic-regression pipeline reaches mean macro-F1 0.899 against 0.907–0.910 for TranscriptFormer variants — and they offer a geometric explanation: the biologically realized cell manifold is well approximated by a **linear subspace**, so once noise is suppressed performance *saturates* and extra expressivity buys little. That is a principled account of why a four-layer MLP over normalized gene space stays competitive, and it matches our own finding that the residual gap to heavier methods was mostly **representation budget, not model class** (§3.9). Their further result that simple methods do *best* out-of-distribution — novel cell types and unseen organisms — also rhymes with the rejection behavior we depend on (§3.7).

Practical guidance. For a **one-off annotation** on laptop-sized data, reach for the **tuned linear pipeline** (normalize $\rightarrow$ ANOVA $\rightarrow$ PCA $\rightarrow$ logistic regression): it is the most accurate-per-second method here and beats actinn-jax outright on accuracy and fit time. If you have a **real atlas and time to train**, **ProtoCloud** is the most accurate method we benchmarked (0.976 at 49k cells), and its GPU target makes its $19\times$ -CPU fit cost far less punishing in practice. **scTOP** is the right tool only for small, low-cardinality problems, where it is dramatically the cheapest (1–2 s) and competitive. **actinn-jax** is the choice when **memory is the binding constraint** ($2.1\times$ lighter than the linear pipeline at scale, with memory-bounded chunked inference), when the reference is **reused** across many queries (the cached model amortizes; the linear pipeline refits scaler/PCA/classifier each time), or when you want the **workflow** — broad $\rightarrow$ refined hand-off, tissue-aware refinement, calibrated abstain, novel-cell-type screening. Of these, the **ready-to-run broad reference** and **sub-second frozen-model refinement** have no counterpart in the baselines; ProtoCloud does provide uncertainty, gene attribution, and a refinement path (by fine-tuning a pretrained model — stronger than masking, but a training run), so we do not claim workflow exclusivity.

5 Limitations

- Single hardware family for the in-house panel (Apple Silicon); no discrete-GPU numbers there (deep/foundation tiers would be faster on CUDA — out of scope for the “runs-on-a-laptop” question). The §3.9 controlled run covers the **CPU tier** on one cloud box; the GPU/R methods there are reported from OP’s own cloud-CI trace (indicative). A same-hardware GPU-tier run to fold those into a single controlled table is the natural extension.
- Subsampled references (to keep the matrix tractable); the scaling section characterizes the size dependence directly.

- Human only; six datasets per benchmark; GPU foundation models beyond scPRINT/UCE (scGPT, Geneformer, popV) not run locally.
- actinn-jax needs more cells/type than linear methods to reach parity (§3.1); on very small references it trails.
- **The gene budget is dataset-dependent, not a free parameter.** More genes help most references but *overfit* a tiny, fine-grained, domain-shifted query (tabula_sapiens −10 pt); our reference-CV + cells-per-class selection rule is validated on 6 datasets with a single clean failure case, so it is directional evidence, not a tuned threshold.
- **Standardization is shipped opt-in**, not default, because it shifts probability calibration that the two-stage abstain thresholds are tuned against; combining the two cleanly would require re-tuning those thresholds.
- **A tuned linear baseline is more accurate than actinn-jax (measured, all four datasets).** Our classical tier (SVM, kNN, CellTypist) is untuned. Adding Souza & Mehta’s pipeline (normalize → ANOVA → standardize → PCA(220) → logistic regression) and **scTOP** (SIMPLE_BASELINES.md) shows the linear pipeline **beats actinn-jax on mean accuracy (0.880 vs 0.867) and macro-F1 (0.860 vs 0.839) while fitting 3.7× faster** — a tie on pbmc3k, slight edges on lung and liver, and **+4.2 pt on the 86-type blood+gut set** (0.902 vs 0.860). It also beats ProtoCloud on 3 of 4. The §3.1 margins therefore reflect *untuned* baselines, and **actinn-jax’s case rests on cost profile and workflow, not accuracy**: bounded sparse memory (1.7 GB vs 3.6 GB mean, 2.6 vs 5.5 GB on blood+gut, and the gap grows because ANOVA/PCA densify cells × genes), the cached reusable reference, and broad→refined / abstain / novel-detection. For one-shot labels on laptop-sized data, the linear pipeline is the better default. scTOP is cheaper still (1.2 s, 1.4 GB) with the best pbmc3k macro-F1 (0.837), but degrades with cardinality (0.910 → 0.828 → 0.795; 0.649 on liver).
- **At atlas scale the deep model wins, and our memory argument is smaller than claimed** (SCALING_MEMORY.md). Scaling the lung reference 3k → 49k cells:
 - (i) **all four methods complete** — we had projected the linear pipeline would need ~32 GB and fail; it used **13.2 GB**, so that projection is retracted; (ii) the linear/actinn-jax memory ratio is **bounded at ~2–3× and does not diverge** (2.15× at full scale), so there is no scaling cliff; (iii) **ProtoCloud goes from worst (0.722 at 3k) to best (0.976 at 49k)**, well clear of actinn-jax (0.936) and the linear pipeline (0.939) — our earlier “accuracy is close” reading was an artifact of subsampling, and a data-hungry VAE pulls ahead once given a real atlas (at 19× CPU fit cost); (iv) scTOP does not benefit from scale at all (0.819 → 0.846) and *loses* its memory edge (1.22× actinn-jax at 49k). **actinn-jax is not the accuracy leader at any scale tested here.**
- **Annotation only.** Cross-species transfer and disease-state prediction — the tasks where Souza & Mehta find the largest foundation-model deficits, and where a fast method would be most attractive — are outside this benchmark’s scope (human, within/cross-dataset annotation).

Data & code availability

All configs (configs/paper*.yaml), adapters (benchmark/adapters/), result CSVs (results/paper*/, docs/results_*.csv), figure scripts, and the actinn-jax package (github.com/iandriver/actinn-jax) are in these two repositories. HLiCA data © Edgar et al. 2026 (CC-BY 4.0), [doi:10.64898/2026.06.30.735539].

References

1. Ma F, Pellegrini M. ACTINN: automated identification of cell types in single cell RNA sequencing. *Bioinformatics* 36(2):533-538 (2020).
2. Abdelaal T, et al. A comparison of automatic cell identification methods for single-cell RNA sequencing data. *Genome Biology* 20:194 (2019).
3. Huang Q, et al. Benchmarking single-cell cell-type annotation methods. *Briefings in Bioinformatics* 25(5):bbae392 (2024).
4. Domínguez Conde C, et al. Cross-tissue immune cell analysis reveals tissue-specific features in humans (CellTypist). *Science* 376:eabl5197 (2022).
5. Aran D, et al. Reference-based analysis of lung single-cell sequencing reveals a transitional profibrotic macrophage (SingleR). *Nature Immunology* 20:163-172 (2019).
6. Kiselev VY, Yiu A, Hemberg M. scmap: projection of single-cell RNA-seq data across data sets. *Nature Methods* 15:359-362 (2018).
7. Xu C, et al. Probabilistic harmonization and annotation of single-cell transcriptomics data with deep generative models (scANVI). *Molecular Systems Biology* 17:e9620 (2021).
8. Lotfollahi M, et al. Mapping single-cell data to reference atlases by transfer learning (scArches). *Nature Biotechnology* 40:121-130 (2022).
9. Chen T, Guestrin C. XGBoost: a scalable tree boosting system. *KDD* 785-794 (2016).
10. Rosen Y, et al. Universal cell embeddings: a foundation model for cell biology (UCE). *Nature* (2026), doi:10.1038/s41586-026-10689-z.
11. Lin Z, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model (ESM-2). *Science* 379:1123-1130 (2023).
12. Open Problems for Single-Cell Analysis Consortium. Open Problems: a living benchmark for single-cell analysis. openproblems.bio (2024).
13. Bradbury J, et al. JAX: composable transformations of Python+NumPy programs (2018). github.com/google/jax.
14. Edgar R, et al. The Human Liver Cell Atlas (HLiCA). doi:10.64898/2026.06.30.735539.
15. Souza H, Mehta P. Parameter-free representations outperform single-cell foundation models on downstream benchmarks. *bioRxiv* (2026), doi:10.64898/2026.02.11.705358.
16. Guo K, Ding J. ProtoCloud: a prototypical self-explaining model for single-cell analysis. *Cell Genomics* 6(6):101217 (2026), doi:10.1016/j.xgen.2026.101217.
17. Yampolskaya M, Souza H, et al. scTOP: cell identity from single-cell data via parameter-free projection. github.com/Emergent-Behaviors-in-Biology/scTOP.